

Aula 7 - Revisão de Java: Lambdas, Records e Streams

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão de Conceitos Básicos (15 min)

- Discussão sobre:
 - Diferenças entre Java 8 e Java 17.
 - Importância da programação funcional no Spring.

2. Expressões Lambda (20 min)

Características

- Sintaxe concisa para implementar interfaces funcionais.
- Exemplo:

```
java

// Antes
Comparator<String> comparator = new Comparator<>() {
    @Override
    public int compare(String s1, String s2) {
        return s1.length() - s2.length();
    }
};

// Com Lambda
Comparator<String> comparator = (s1, s2) -> s1.length() - s2.length();
```

Interfaces Funcionais Comuns

Interface	Método	Uso
Predicate<T>	test(T t)	Filtros (ex: Streams)
Function<T,R>	apply(T t)	Transformações de dados

Interface	Método	Uso
Consumer<T>	accept(T t)	Ações sem retorno

3. Records (15 min)

- Classes imutáveis para transportar dados (DTOs).
- Exemplo:

```
java

// Antes (Java <14)
public class User {
    private final String name;
    private final int age;
    // Construtor, getters, equals, hashCode, toString...
}

// Com Record (Java 14+)
public record User(String name, int age) {}
```

4. Streams API (10 min)

- Operações em coleções com estilo funcional:

```
java

List<String> names = List.of("Ana", "Carlos", "Maria");
List<String> filtered = names.stream()
    .filter(name -> name.length() > 3)
    .map(String::toUpperCase)
    .toList();
```

Parte Prática (3 horas)

Atividade: Sistema de Gerenciamento de Produtos

Passo 1: Implementar Records (45 min)

1. Crie records para modelagem:

```
java

public record Product(String id, String name, double price, int stock) {}
```

```
public record Order(String id, List<Product> products, LocalDate date) {}
```

2. Uso básico no main :

```
java
```

```
Product p1 = new Product("P1", "Notebook", 4500.0, 10);  
System.out.println(p1.name()); // Acessor automático
```

Passo 2: Operações com Streams (1h)

1. Crie uma lista de produtos:

```
java
```

```
List<Product> products = List.of(  
    new Product("P1", "Mouse", 50.0, 15),  
    new Product("P2", "Teclado", 120.0, 8),  
    new Product("P3", "Monitor", 800.0, 3)  
);
```

2. Implemente operações:

```
java
```

```
// 1. Filtre produtos com estoque < 10  
List<Product> lowStock = products.stream()  
    .filter(p -> p.stock() < 10)  
    .toList();  
  
// 2. Calcule o valor total do estoque  
double totalValue = products.stream()  
    .mapToDouble(p -> p.price() * p.stock())  
    .sum();  
  
// 3. Mapeie nomes em maiúsculas  
List<String> upperCaseNames = products.stream()  
    .map(Product::name)  
    .map(String::toUpperCase)  
    .toList();
```

Passo 3: Lambdas em Interfaces Funcionais (1h)

1. Crie um serviço com Predicate :

```
java
```

```
public class ProductService {  
    public List<Product> filterProducts(List<Product> products, Predicate<Product> filter) {  
        return products.stream().filter(filter).toList();  
    }  
}
```

```

    }
}

// Uso:
ProductService service = new ProductService();
List<Product> expensive = service.filterProducts(products, p -> p.price() > 100);

```

2. Use Function para transformações:

```

java

Function<Product, String> toDescription = p ->
    String.format("%s - R$%.2f (Estoque: %d)", p.name(), p.price(), p.stock());

List<String> descriptions = products.stream()
    .map(toDescription)
    .toList();

```

Passo 4: Integração com Spring Boot (15 min)

1. Converta ProductService para um @Service Spring.

2. Injete o serviço em um @RestController:

```

java

@RestController
@RequestMapping("/products")
public class ProductController {
    @Autowired
    private ProductService service;

    @GetMapping("/expensive")
    public List<Product> getExpensiveProducts() {
        return service.filterProducts(products, p -> p.price() > 100);
    }
}

```

Tarefa para Casa

1. Implemente Comparator com lambda:

- Ordene produtos por preço (decrecente) e depois por nome.

2. Desafio com Collectors:

- Agrupe produtos por faixa de preço (ex: <100, 100-500, >500).

Próxima Aula

- **Tópico:** Spring Data JPA com Java 17.
- **Atividade prática:** CRUD com Repository e Records.

Resultado Esperado

- Domínio modelado com Records.
- Operações funcionais com Streams e Lambdas.
- Integração básica com Spring Boot.

java

```
// Exemplo de saída no console:  
Product[id=P1, name=Mouse, price=50.0, stock=15]  
Valor total do estoque: R$ 9850.00  
Produtos com estoque baixo: [Teclado, Monitor]
```

<https://www.baeldung.com/wp-content/uploads/2018/11/Java-Streams.jpg> (Pipeline de operações com Streams) 🚀

